

UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Grupos de Investigación Multimedia Interactiva

CODEMICHI

Game Design Document (GDD)

Aprendiendo a programar mediante videojuegos

Equipo	Yader Ibraldo Quiroga Torres - 20222020034 Jhonathan De La Torres - 20222020033
Versión	1.0 — Borrador
Fecha de entrega	22/09/2026

Tabla de contenido 2

1. Resumen del juego 3
1.1 Concepto general 3
1.2 Género y objetivo pedagógico 3
1.3 Público objetivo 3
1.4 Plataforma y alcance técnico 3
2. Historia y narrativa 3
2.1 Premisa 3
2.2 Estructura narrativa 4
2.3 Personajes 4
3. Mecánicas de juego 4
3.1 Mecánica principal (programación por bloques/consola) 4
3.2 Reglas y condiciones de victoria/derrota 4
3.3 Sistema de progresión 5
3.4 Controles e interacción 5
4. Diseño de niveles 5
4.1 Estructura general 5
4.2 Niveles iniciales (tutorial) 6
4.3 Progresión de dificultad 6
5. Arte y estilo visual 6
5.1 Dirección de arte 6
5.2 Personajes y assets 6
5.3 UI / UX visual 6
6. Sonido 7
6.1 Música (OST) 7
6.2 Efectos de sonido (SFX) 7
7. Tecnología y herramientas 7
7.1 Motor de desarrollo 7
7.2 Herramientas de arte y sonido 7
8. Referencia al plan de trabajo 7

1. Resumen del juego

1.1 Concepto general

Elevator pitch en 2-3 frases: qué es el juego y qué lo hace distinto.

Codemichi es un videojuego de plataformas en pixel art en el que un gato minero debe programar sus propias estrategias para abrirse paso a través de una mina compuesta de retos, cada nivel está diseñado alrededor de un concepto de programación (secuencias, bucles, condicionales) que el jugador aplica directamente escribiendo o formando bloques de código para resolver el reto y avanzar, a diferencia de otros juegos educativos, aquí el código no es un ejercicio aparte: es la principal y única forma de jugar.

1.2 Género y objetivo pedagógico

Género Principal:Puzzle Programable.
Subgéneros: Programación Visual / Educativo.
Perspectiva:2D Top-Down.
Objetivo: Enseñar sobre conceptos básicos de programación mediante bloques de programación.

1.3 Público objetivo

Edad, nivel previo de conocimiento en programación, contexto de uso (aula, autoestudio).

Codemichi está dirigido a estudiantes de primer semestre universitario y recién graduados de colegio, entre el rango de edad de 16 y 19 años que se están acercando a la programación por primera vez o que apenas tienen nociones básicas, está pensado para un contexto de aula (como apoyo en cursos introductorios de programación o lógica de programación) y también para autoestudio sin que el jugador necesite conocimientos previos de sintaxis de ningún lenguaje, el tono y el ritmo del juego asumen que quien juega puede sentirse intimidado por el código, por lo que prioriza la experimentación y el ensayo y error por encima de la memorización de reglas.

1.4 Plataforma y alcance técnico

Dispositivo: PC (Windows) y web (HTML5)
Motor/Engine: Godot Engine 4.6.1
Duración estimada por partida: 30 a 45 minutos en completar los niveles iniciales de este demo.

2. Historia y narrativa

2.1 Premisa

Contexto: Codemichi, el gato minero que enseña a programar mientras explora la mina.

Codemichi es un gato minero que ha heredado una vieja mina abandonada repleta de minerales, para avanzar Code Michi no usa un pico sino que usa código escribiendo instrucciones que le sirven para abrirse camino, el jugador acompaña a Codemichi en este descenso aprendiendo a programar.

2.2 Estructura narrativa

Primer Mundo: Galerias Superficiales (Parte Secuencial).*Mina de entrada* , el Michi usa bloques básicos para recolectar minerales basicos y desbloquear nuevos bloques

- Usar variantes del mundo para diversos bloques básicos

Segundo Mundo: Vías del vagón (Bucles) Automatizar los movimientos mediante movimientos repetitivos y minería continua.

- Variantes con vías y minerales más raros.

Tercer Mundos: Mundo Cristal. Usar bloques de bucles y condicionales para moverse por un mundo con más peligros y recompensas más valiosas.

- Variantes del mundo con diversos retos y posibles trampas.

En este mundo el Michi encontrará la salida de la mina.

2.3 Personajes

Ficha por personaje: nombre, rol, personalidad, relación con el jugador.

Codemichi — Protagonista: Gato minero, es el avatar del jugador, cada línea de código que escribe la ejecuta Codemichi en la mina.

Vieja Nano — Mentora: Lámpara minera con una IA antigua que da pistas y explica los conceptos nuevos sin resolver los retos al jugador.

3. Mecánicas de juego

3.1 Mecánica principal (programación por bloques/consola)

Cómo el jugador escribe o arma instrucciones y cómo se ejecutan en el mundo del juego.

La mecánica central es un editor de programación visual por bloques inspirado en Scratch a un lado de la pantalla hay una paleta de bloques organizados por categoría y color (movimiento, bucles, condicionales, eventos, acciones sobre la mina) y el jugador los arrastrar y encajar como piezas de rompecabezas para armar un programa en un panel de trabajo.

Cada nivel plantea un reto concreto en la mina ya sea para obtener los minerales o para avanzar en el entorno que solo se resuelve con una combinación específica de bloques, cuando el jugador presiona "Ejecutar" Codemichi corre el programa en tiempo real dentro del nivel, los bloques se van resaltando uno a uno a medida que se ejecutan, para que el jugador pueda seguir el flujo de su propio código, si el programa falla la mina lo muestra de forma visual en vez de tirar un mensaje de error técnico y el jugador puede ajustar los bloques y volver a ejecutar tantas veces como quiera.

3.2 Reglas y condiciones de victoria/derrota

Condición de Victoria: Michi alcanza la casilla de salida/gema del nivel ejecutando el programa completo con los bloques de programación disponibles.

Condición de Derrota: Michi no alcanza la meta (gemas), choca contra un peligro (Bloqueos en el mapa) o el programa termina y Michi no ha llegado a la meta. Al fallar, el nivel se reinicia sin perder el código escrito para permitir el ajuste (*debugging*).

3.3 Sistema de progresión

Curva de dificultad: qué conceptos (secuencia, bucles, condicionales) se introducen y en qué orden.

La curva de dificultad avanza por concepto de programación, cada mundo de la mina introduce una única estructura nueva (secuencia, bucles, condicionales, variables) y la combina con lo aprendido en mundos anteriores, así el jugador es capaz de enlazar los conceptos.

Los primeros mundos se limitan a secuencias simples de movimiento para que el jugador se familiarice con arrastrar y ejecutar bloques sin lógica de por medio, después se introducen ciclos para resolver retos repetitivos mostrando de forma tangible por qué repetir bloques a mano es ineficiente, después se introducen condicionales obligando a Codemichi a "decidir" según el estado de la mina, los mundos siguientes combinan estas tres estructuras en retos cada vez más largos, y solo en los niveles finales se exige empaquetar y anidar lógica en funciones.

3.4 Controles e interacción

- Mouse: Arrastrar y soltar bloques de código (Drag & Drop) en la interfaz del usuario (UI)
- Teclado: Atajos rápido (usar teclas como espacio para iniciar/ detener el código , R para reiniciar el nivel etc..)

4. Diseño de niveles

4.1 Estructura general

Cantidad de niveles/mundos y cómo se agrupan por concepto de programación enseñado.

La mina se organiza en rooms y cada room es un nivel, cada room está atada a un concepto de programación puntual y ese concepto se trabaja a lo largo de varias rooms consecutivas con retos cada vez un poco más exigentes antes de pasar al concepto siguiente.

El orden de los conceptos sigue la misma progresión que el sistema de progresión: secuencia, ciclos, condicionales, funciones.

- **Rooms de Secuencias:** varias rooms centradas en movimiento simple, pensadas para que el jugador se familiarice con arrastrar y ejecutar bloques sin lógica de por medio.
- **Rooms de Ciclos:** varias rooms con retos repetitivos donde repetir bloques a mano se vuelve evidentemente ineficiente, motivando el uso de ciclos.
- **Rooms de Condicionales:** varias rooms donde Codemichi debe "decidir" según el estado de la mina combinando condicionales con la secuencia y los ciclos ya aprendidos.
- **Rooms siguientes (Combinación y funciones):** rooms más largas que combinan las tres estructuras anteriores en retos compuestos, hasta llegar a rooms donde se exige empaquetar y anidar lógica en funciones para mantener el programa manejable.

4.2 Niveles iniciales (tutorial)

- Nivel 1.1 (primeros pasos)
    - Objetivo: alcanzar 3 casillas
    - Bloques Disponibles: Avanzar
    - Aprendizaje:Comprender que la ejecución es secuencial de arriba a abajo.
- Nivel 1.2 (Mina con mineral)
    - Objetivo : Avanzar, evitar obstáculos y picar minerales.
    - Aprendizaje:identificar la diferencia entre diversas acciones.

4.3 Progresión de dificultad

Cómo escala la complejidad entre niveles.

Dentro de cada bloque de rooms de un mismo concepto la dificultad no escala añadiendo conceptos nuevos (eso ya lo cubre el paso de un concepto a otro) sino ajustando cuatro variables sobre el mismo concepto:

- **Complejidad del reto:** las primeras rooms de un concepto tienen un solo obstáculo que resolver con él y una forma simple de extraer los minerales; las últimas encadenan varios obstáculos del mismo tipo en un mismo recorrido.
- **Restricciones sobre la solución:** rooms más avanzadas limitan el número de bloques disponibles o de bloques que se pueden usar, obligando al jugador a encontrar una solución más eficiente en vez de una que simplemente funcione.
- **Nivel de ayuda:** las primeras rooms de cada concepto muestran pistas de Vieja Nano de forma proactiva; en las rooms finales del bloque las pistas hay que pedirlas explícitamente, o desaparecen.

5. Arte y estilo visual

5.1 Dirección de arte

Estilo (pixel art), paleta de colores, referentes visuales.

La dirección de arte es pixel art con una paleta de colores cálida y saturada que contrasta la piedra oscura de la mina con los tonos brillantes de los cristales de modo que el jugador pueda distinguir de un vistazo qué elementos de la room son interactuables, el estilo se mantiene deliberadamente sencillo y legible sprites de bajo detalle, siluetas claras, poca ambigüedad visual porque el público objetivo debe poder identificar en segundos qué hace cada objeto sin que el arte compita con la lógica del reto.

Cada concepto de programación tiene una asociación visual consistente a lo largo de todas sus rooms, para reforzar el aprendizaje también por reconocimiento, las mecánicas de secuencia, los ciclos se representan, los condicionales y las funciones se representan con usan vagonetas y rieles, bifurcaciones de rieles o compuertas.

5.2 Personajes y assets

Personajes Principales:

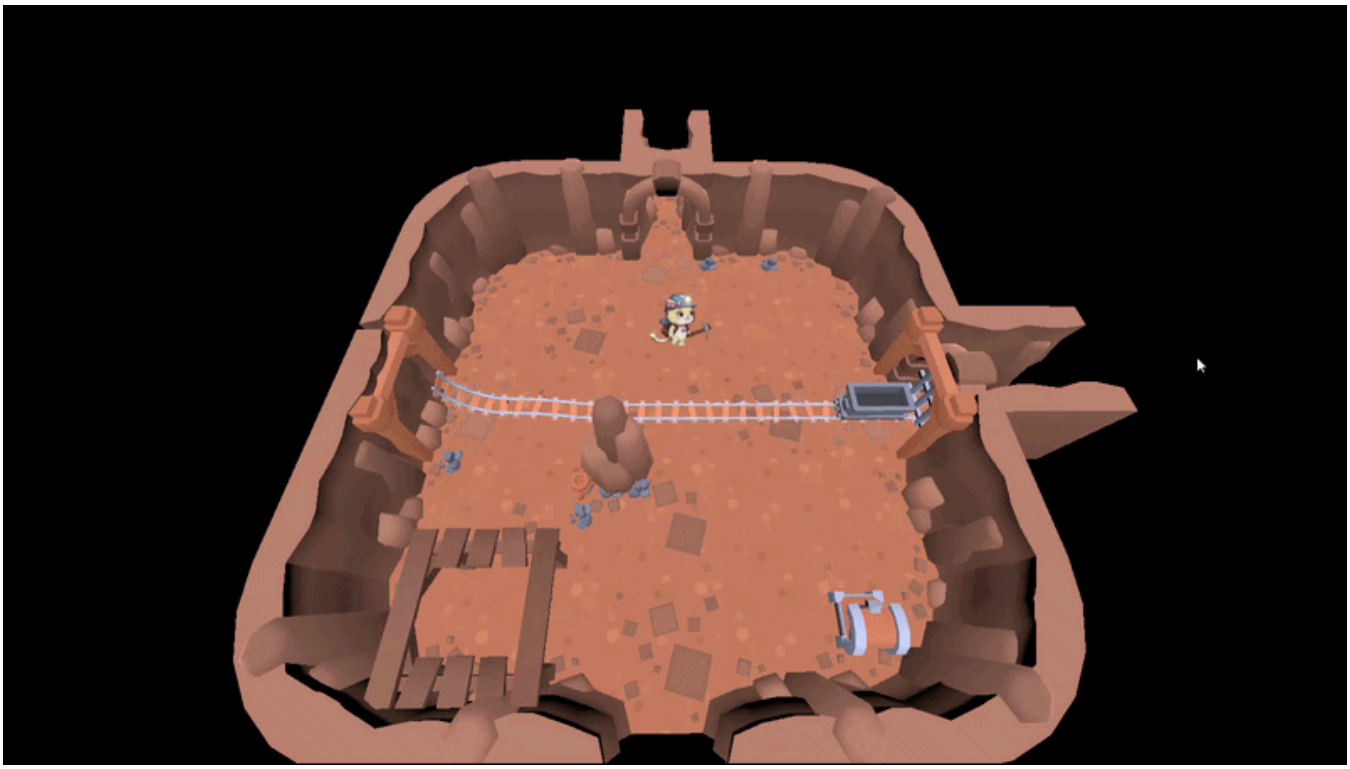
Gato Macho (Michi)



Gata Hembra (Michelle)



Para los mapas usaremos un estilo de habitaciones con cambios de mapas con Vagones.





El objetivo es usar mapas que representen habitaciones para cambiar de dificultad.

5.3 UI / UX visual

Look and feel del HUD, menús y editor de código/bloques.

La pantalla se divide en dos zonas fijas: el panel de bloques (paleta y área de ensamblaje) y la room de la mina, donde se ve en vivo lo que hace el programa, así el jugador nunca pierde de vista la relación entre el bloque que arrastra y lo que pasa en el juego, los bloques usan el código de colores por categoría típico de Scratch, y esos mismos colores se repiten en los elementos de la room (rieles, engranajes, compuertas), reforzando la asociación visual de cada concepto, el botón "Ejecutar" es el elemento más prominente de la pantalla, siempre en el mismo lugar.

6. Sonido

6.1 Música (OST)

Ambientación sonora por mundo/nivel.

de stock

6.2 Efectos de sonido (SFX)

Sonidos de acciones clave: ejecutar código, error, victoria.

Los efectos de sonido saldrán de una biblioteca de acceso gratis de la página de kenney

7. Tecnología y herramientas

7.1 Motor de desarrollo

Engine/lenguaje elegido y justificación.

El motor elegido es Godot por tres razones puntuales al proyecto: tiene un motor 2D nativo lo que facilita el pixel art y el manejo de las habitaciones, es gratuito y de código abierto, sin costos de licencia ni de regalías relevante para un proyecto estudiantil y su sistema de nodos y señales se presta bien para construir algo tan particular como el editor de bloques arrastrables y su ejecución paso a paso dentro de la room.

7.2 Herramientas de arte y sonido

Software usado para producir los assets.

El software elegido para el desarrollo del estilo gráfico se trata Makebead para la creación del pixel art de los escenarios, personajes y iconos

8. Referencia al plan de trabajo

Enlazar o resumir el cronograma de actividades (documento "plan de trabajo") y responsables por sección.

plan de trabajo